

UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES
DEPARTAMENTO DE COMPUTACIÓN

SISTEMAS OPERATIVOS I

Trabajo Práctico Nro 1: "Activación del Sector"

Misión: Activar NEXUS-9

Grupo de Trabajo:

Grupo Infostealer

Integrantes:

- Kevin Gonzalo Cufre Quiroz
- Rocco Ignacio Delgado Velazquez
- Facundo Luis Lugones Oviedo

Cuerpo Docente:

Ing. Hugo Carrer
Gabriel Valenzuela
Pablo Martinez
Nicolas Melia

Fecha:

Octubre de 2026

Córdoba, República Argentina

0. Introduccion y Contexto de la Misión

El presente informe documenta la ejecución del **Trabajo Práctico N° 1: "Activación del Sector"**, correspondiente a la misión **Activar NEXUS-9**.

El objetivo principal consiste en diseñar, desplegar, interconectar y asegurar una infraestructura distribuida compuesta por dos nodos computacionales basados en distribuciones GNU/Linux de familias distintas (Debian y Red Hat Enterprise Linux), abordando:

- Proveyendo de infraestructura virtual y configuración de red.
- Resolución de nombres estática y verificación de enlace en Capa 3.
- Gestión de identidades, jerarquía de grupos y control de privilegios (DAC, `sudo`, Principio de Mínimo Privilegio).
- Acceso remoto seguro mediante criptografía asimétrica SSH-2 (Ed25519) y *hardening* del servicio.
- Desarrollo de scripts de automatización e inventariado agnósticos a la distribución.
- Análisis profundo y experimentación práctica sobre redes locales (Capa 2 vs. Capa 3, protocolo ARP/NDP, ciclo de vida de entradas y colisiones de hardware).

1. Etapa 01 — Infraestructura del Sector

Para conformar el sector NEXUS-9, se desplegaron dos máquinas virtuales (VMs) independientes sobre VirtualBox, asignando a cada una un rol diferenciado:

1. **sector-core**: Servidor central de cómputo y coordinación. Implementado sobre **Ubuntu Server 26.04.1 LTS** (Kernel 7.0.0-30-generic).
2. **field-terminal**: Estación de trabajo de operadores en campo. Implementada sobre **Rocky Linux 10.2 (Red Quartz)** (Kernel 6.12.0-211.16.1.el10_2.0.1.x86_64).

Tabla 1: Parámetros y especificaciones técnicas de los nodos del sector NEXUS-9.

Parámetro	sector-core	field-terminal
Sistema Operativo	Ubuntu Server 26.04.1 LTS	Rocky Linux 10.2 (Red Quartz)
Familia / Base	Debian GNU/Linux	Red Hat Enterprise Linux (RHEL)
Rol Asignado	Servidor principal del sector	Terminal de operadores en campo
Versión de Kernel	7.0.0-30-generic	6.12.0-211.16.1.el10_2.0.1.x86_64
Interfaz NAT (`enp0s3`)	10.0.2.15/24 (Salida WAN)	10.0.2.15/24 (Salida WAN)
Interfaz Privada (`enp0s8`)	192.168.56.10/24	192.168.56.20/24
IP de Laboratorio / Enlace	172.18.113.147	172.18.113.x

Topología de Red de las Máquinas Virtuales

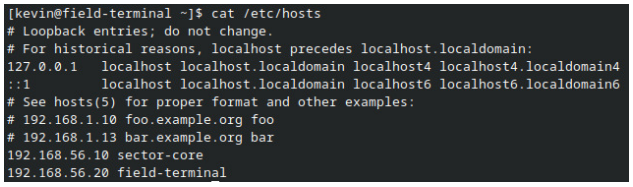
Cada nodo cuenta con dos placas de red: **`enp0s3`** en modo NAT (acceso externo) y **`enp0s8`** en modo Red Interna / Host-Only (`192.168.56.0/24`), garantizando enlace punto a punto y aislamiento de tráfico de Capa 2.

2. Etapa 02 — Comunicación entre las Máquinas

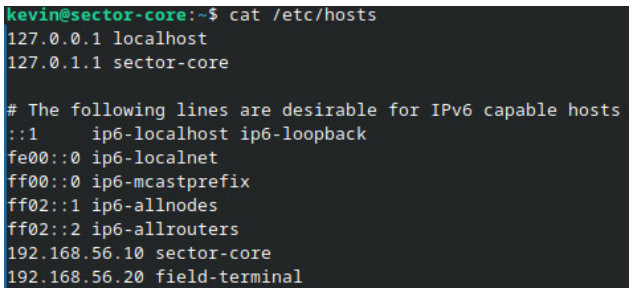
2.1. Resolución Estática de Nombres mediante `/etc/hosts`

En entornos aislados sin servidor DNS, la resolución de nombres se configuró estaticamente editando `/etc/hosts` en ambas VMs. El archivo `/etc/nsswitch.conf` establece la directiva `hosts: files dns`, lo que hace que el sistema operativo consulte `/etc/hosts` de forma inmediata antes de recurrir a consultas por red.

En cada máquina se agregaron dos entradas al final del archivo: la IP de la red interna de `sector-core` (`192.168.56.10`) asociada al nombre `sector-core`, y la IP de `field-terminal` (`192.168.56.20`) asociada al nombre `field-terminal`. De esta forma, ambas VMs pueden resolver el nombre de la otra sin necesidad de un servidor DNS externo.



(a) Archivo `/etc/hosts` en Rocky Linux (`field-terminal`): entradas para `sector-core` y `field-terminal` en la red `192.168.56.0/24`.



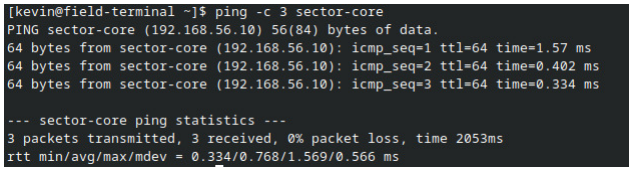
(b) Archivo `/etc/hosts` en Ubuntu Server (`sector-core`): mismas entradas simétricas configuradas.

Figura 1: Resolución estática de nombres configurada en ambas VMs mediante `/etc/hosts`.

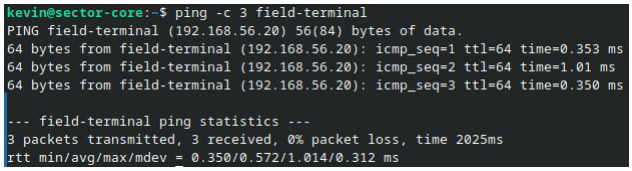
2.2. Verificación Experimental de Conectividad (ICMP Ping)

Una vez configurada la resolución de nombres, se verifico la conectividad bidireccional en Capa 3 mediante `ping -c 3`. La prueba resuelve el nombre del destino usando `/etc/hosts` y envia 3 paquetes ICMP Echo Request, comprobando que ambas VMs se alcanzan mutuamente a través de la red interna:

- **Rocky → Ubuntu**: `ping -c 3 sector-core` desde `field-terminal` resolvió a `192.168.56.10` con **0 % packet loss** y RTT promedio de **0.768 ms**.
- **Ubuntu → Rocky**: `ping -c 3 field-terminal` desde `sector-core` resolvió a `192.168.56.20` con **0 % packet loss** y RTT promedio de **0.572 ms**.



(a) `ping -c 3 sector-core` desde Rocky Linux: 0 % packet loss, RTT avg 0.768 ms.



(b) `ping -c 3 field-terminal` desde Ubuntu: 0 % packet loss, RTT avg 0.572 ms.

Figura 2: Verificación de conectividad bidireccional en Capa 3 entre `field-terminal` y `sector-core`.

3. Etapa 03 — Usuarios, Grupos y Privilegios

3.1. Fundamentación Teórica

Pregunta: ¿Qué diferencia fundamental existe entre un usuario y un grupo?

Respuesta: Un **usuario** representa una identidad individual en el sistema asociada a un identificador entero único (**UID**). Puede representar una persona física o un demonio del sistema. Un **grupo** es una entidad colectiva identificada por un **GID**, utilizada para aplicar políticas de permisos concurrentes sobre archivos y procesos en el esquema DAC (*Discretionary Access Control*).

Pregunta: ¿Qué información contienen `/etc/passwd`, `/etc/group` y `/etc/shadow`?

Respuesta:

- **`/etc/passwd`** (permisos `644`): Registro de cuentas estructurado como `username:x:UID:GID:GECOS:home:shell`. La `x` indica que el hash de la clave se ubica en `shadow`.
- **`/etc/group`** (permisos `644`): Define composición de grupos: `group_name:x:GID:user_list`.
- **`/etc/shadow`** (permisos `600`): Información crítica de autenticación: `username:hash:last_change:min:max:warn:inact:expire`. Almacena hashes con *salt* (SHA-512 / Yescrypt).

Pregunta: ¿Qué representan el UID y el GID en la arquitectura del sistema?

Respuesta: Son valores numéricos enteros utilizados por el kernel Linux en la estructura de credenciales de procesos (`struct cred`) para evaluar permisos de acceso en llamadas al sistema (`open`, `read`, `write`, `kill`).

Pregunta: ¿Qué diferencia existe entre el grupo primario y los grupos suplementarios?

Respuesta: El **grupo primario** define el GID propietario por defecto de los archivos creados por el usuario en su sesión interactiva. Los **grupos suplementarios** son membresías adicionales para acceder a recursos compartidos (como `sudo`, `wheel` o `docker`).

Pregunta: ¿Por qué el usuario root posee un tratamiento especial?

Respuesta: `root` posee el **UID = 0**. El kernel omite las comprobaciones habituales de permisos cuando el UID efectivo es 0, otorgándole acceso irrestricto sobre recursos de hardware, memoria y sistema de archivos.

Pregunta: ¿Qué problemática intenta resolver el comando `sudo`?

Respuesta: Permite ejecutar comandos privilegiados de forma granular sin divulgar la contraseña de `root`. Ofrece autenticación individual, control fino en `/etc/sudoers` y auditoría completa en los logs (`/var/log/auth.log` en Ubuntu o `/var/log/secure` en Rocky).

Pregunta: ¿Qué implica aplicar el Principio de Mínimo Privilegio (PoLP)?

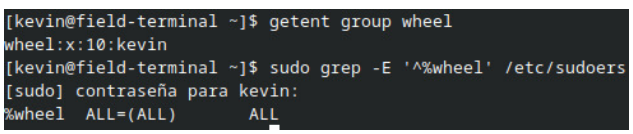
Respuesta: Exige conceder a cada usuario o servicio únicamente los privilegios estrictamente necesarios para su operación, reduciendo la superficie de ataque y el impacto de errores operativos o compromisos de seguridad.

3.2. Configuración Aplicada

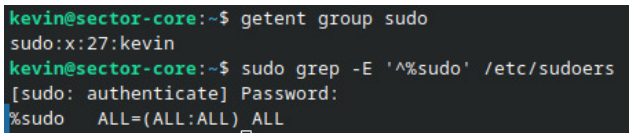
En ambos nodos se creó la cuenta de administración individual `kevin`, asignada al grupo administrativo correspondiente a cada distribución: `wheel` en Rocky Linux (`field-terminal`) y `sudo` en Ubuntu Server (`sector-core`). Esta elección respeta el principio de mínimo privilegio: el usuario no opera como `root` en su sesión habitual, pero puede elevar privilegios cuando es necesario mediante `sudo`.

Para verificar y documentar la configuración se ejecutaron dos comandos en cada maquina:

- `getent group wheel` / `getent group sudo` — consulta la base de datos de grupos del sistema (unificando `/etc/group` con fuentes NSS). Permite confirmar qué usuarios pertenecen al grupo administrativo sin editar archivos directamente.
- `sudo grep -E '^%wheel' /etc/sudoers` / `sudo grep -E '^%sudo' /etc/sudoers` — extrae del archivo de política `sudoers` la regla que otorga privilegios completos al grupo. La salida `%wheel ALL=(ALL) ALL` (Rocky) y `%sudo ALL=(ALL:ALL) ALL` (Ubuntu) confirman que cualquier miembro del grupo puede ejecutar cualquier comando como cualquier usuario, satisfaciendo los requisitos de administración delegada.



(a) Verificación en Rocky Linux (`field-terminal`): `kevin` en el grupo `wheel` y regla `%wheel ALL=(ALL) ALL` en `sudoers`.



(b) Verificación en Ubuntu Server (`sector-core`): `kevin` en el grupo `sudo` y regla `%sudo ALL=(ALL:ALL) ALL` en `sudoers`.

Figura 3: Documentación de los usuarios con privilegios administrativos delegados en ambas distribuciones.

4. Etapa 04 — Acceso Remoto Seguro mediante SSH

4.1. Investigación Previa

Pregunta: ¿Qué problema resuelve SSH frente a protocolos tradicionales?

Respuesta: Reemplaza protocolos no seguros (como Telnet o FTP) que transmiten credenciales y datos en texto plano. SSH-2 cifra el canal íntegro mediante algoritmos simétricos robustos (AES, ChaCha20) tras un intercambio Diffie-Hellman autenticado.

Pregunta: ¿Diferencia entre autenticación por contraseña y por clave publica?

Respuesta: La contraseña es vulnerable a fuerza bruta y viaja por el túnel cifrado hacia el servidor. En la autenticacion por clave publica, el servidor emite un desafio aleatorio (*nonce*) que el cliente firma con su clave privada; la clave privada jamás abandona el equipo local.

Pregunta: ¿Función de la clave privada, clave publica y archivo `authorized_keys`?

Respuesta: La **clave privada** firma desafíos para certificar identidad; la **clave pública** permite al servidor verificar la firma. El archivo `~/.ssh/authorized_keys` (permisos `600`) en el servidor almacena las claves públicas autorizadas. La clave privada nunca debe residir en el servidor para evitar escaladas laterales si el host es comprometido.

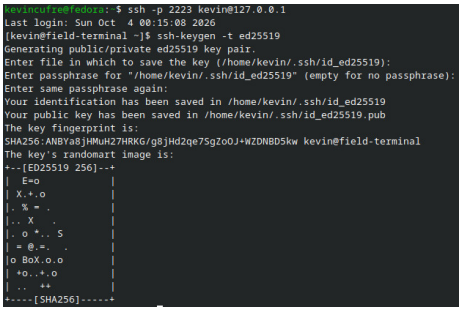
Pregunta: ¿Qué riesgo implica permitir acceso remoto directo como root?

Respuesta: Ofrece un vector fijo y predecible para ataques automatizados de fuerza bruta y anula la trazabilidad de auditoría en los registros del sistema.

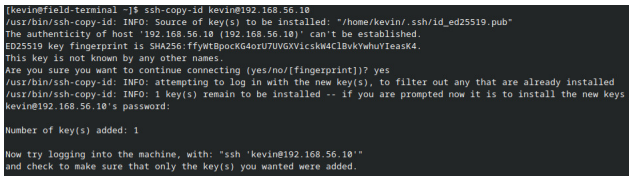
4.2. Generación y Distribución de Claves (Ed25519)

La consigna establece que `field-terminal` (Rocky Linux) debe poder administrar `sector-core` (Ubuntu Server) mediante SSH con autenticación por clave pública. Para ello, el par de claves se generó **dentro de la propia VM Rocky Linux**, que actúa como cliente SSH, y la clave pública fue distribuida hacia Ubuntu a través de la red interna compartida entre ambas VMs (`192.168.56.0/24`).

Desde una sesión ya iniciada en Rocky Linux se ejecutó `ssh-keygen -t ed25519`, generando el par de claves Ed25519 almacenado en `/home/kevin/.ssh/id_ed25519` y `id_ed25519.pub`. A continuación se usó `ssh-copy-id kevin@192.168.56.10` para registrar la clave pública en el archivo `authorized_keys` de Ubuntu, utilizando directamente la IP interna de la red entre las VMs.



(a) Generación del par de claves Ed25519 desde Rocky Linux (`field-terminal`).



(b) Distribución de la clave pública a Ubuntu (`192.168.56.10`) con `ssh-copy-id`.

Figura 4: Generación en Rocky y despliegue de la clave pública hacia Ubuntu a traves de la red interna de las VMs.

4.3. Verificación de la Conexión VM a VM mediante SSH

Una vez registrada la clave pública en Ubuntu, se verificó la conexión directa entre las VMs: desde Rocky Linux se ejecutó `ssh kevin@192.168.56.10`, estableciéndose la sesión sin requerir contraseña. La autenticación se resolvió exclusivamente mediante el par de claves Ed25519. El banner de bienvenida de Ubuntu Server confirma que la conexión fue exitosa y que `field-terminal` puede administrar `sector-core` de forma remota y segura a través de la red interna.

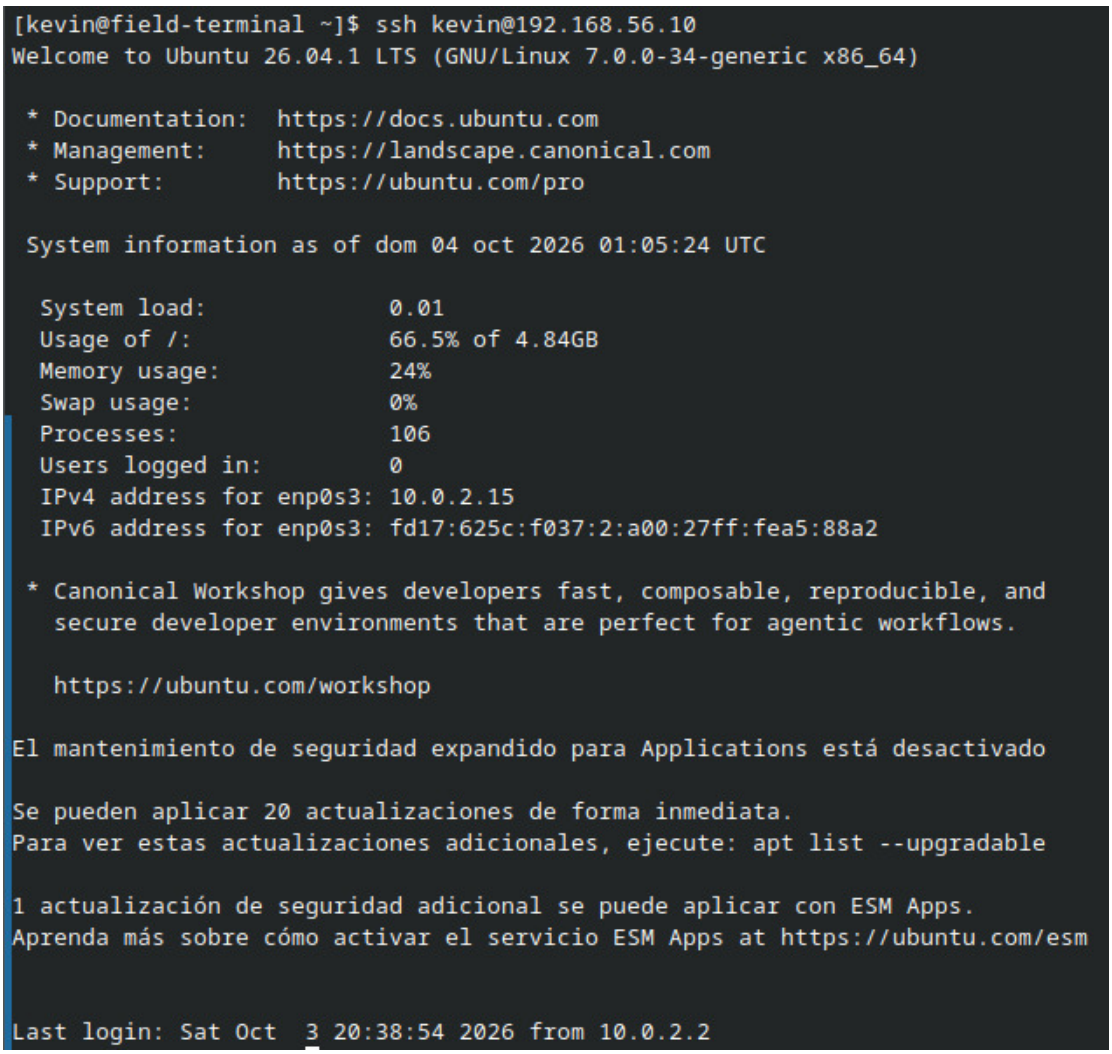
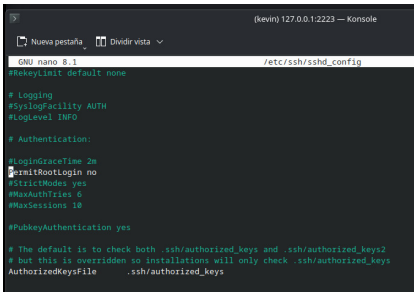


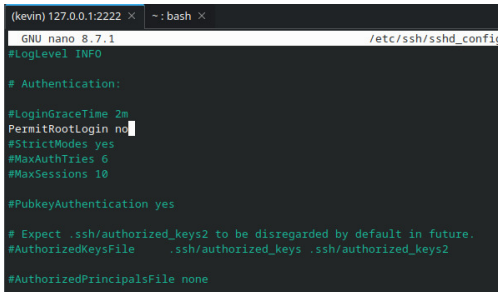
Figura 5: Conexión SSH exitosa desde Rocky Linux (`field-terminal`) hacia Ubuntu Server (`sector-core`, `192.168.56.10`) por la red interna de las VMs.

4.4. Hardening: Deshabilitación del Login Remoto de Root

En `/etc/ssh/sshd_config` de ambos sistemas se configuró `PermitRootLogin no` y `PubkeyAuthentication yes`. Posteriormente se reiniciaron los servicios: `systemctl restart ssh` (Ubuntu) y `systemctl restart sshd` (Rocky).



(a) `sshd_config` en Rocky Linux (puerto 2223).



(b) `sshd_config` en Ubuntu Server (puerto 2222).

Figura 6: Deshabilitación de acceso remoto para `root` en ambas distribuciones.

5. Etapa 05 — Inventario Automatizado del Sistema

5.1. Diseño del Script y Política de Manejo de Errores

Para cumplir con la consigna de relevamiento del sistema se desarrolló el script `system-inventory.sh` en lenguaje Bash. El script fue concebido bajo dos requerimientos primordiales: **agnosticismo respecto**

a la distribución (compatibilidad plena entre Debian/Ubuntu Server y RHEL/Rocky Linux sin codificación de rutas propietarias) y **tolerancia y supervivencia ante fallos** mediante una política de control de errores.

5.1.1. Agnosticismo de Distribución e Introspección Segura

El script no asume utilidades ni rutas exclusivas de una única familia de sistemas operativos. En lugar de verificar archivos específicos como `/etc/debian_version` o `/etc/redhat-release`, implementa estandares abiertos y mecanismos de introspección defensiva:

- **Estándar `/etc/os-release`**: Carga de manera controlada el archivo estándar de especificación `os-release`, extrayendo las variables `PRETTY_NAME` y `VERSION_ID` con mecanismos de sustitución por omisión (`${PRETTY_NAME:-desconocida}`), complementado con las llamadas estándar POSIX `uname -r` (kernel) y `uname -m` (arquitectura).
- **Verificación previa de herramientas (`command_exists`)**: Mediante la función auxiliar:

  ```
  command_exists() { command -v "$1" >/dev/null 2>&1; }
  ```

  se comprueba la existencia de cada binario (`lscpu`, `free`, `lsblk`, `df`, `mountpoint`, `ip`, `systemctl`) antes de su invocación. De este modo, si una utilidad no estuviera disponible en una instalación minimalista, el script notifica el error por el canal de diagnóstico estándar (`stderr`) y retorna un código de error de sección sin provocar abortos inesperados.
- **Procesamiento de opciones de línea de comandos**: Implementa la función `usage()` y procesa banderas mediante la primitiva `getopts ":h"`. Si se invoca con `-h`, despliega la sintaxis y ayuda finalizando limpiamente con código `EXIT_OK` (0). Si se provee un argumento desconocido, reporta la opción inválida por `stderr`, emite la ayuda y finaliza con código de error sintáctico `EXIT_ARGS` (1).

5.1.2. Política de Supervivencia y Manejo de Errores (`set -euo pipefail`)

La consigna plantea una pregunta clave de ingeniería de sistemas: *¿ante la falla de una consulta, el script debe continuar, detenerse o ignorar el error?* La decisión de diseño adoptada fue la de **supervivencia con degradación informada**: ante el fallo de un componente, el script debe continuar recolectando el resto de las secciones, pues **un inventario parcial es significativamente más valioso para la administración que un script que aborta a mitad de camino sin producir resultados**. Sin embargo, el error nunca debe ser ignorado: debe registrarse en `stderr` y reflejarse en el código de salida final del proceso.

Para implementar esta política se recurre a la directiva estricta de Bash `set -euo pipefail`:

- **`-e` (*errexit*)**: Provoca que aborte la ejecución de una función si algún comando retorna un código distinto de 0. Para permitir que el script sobreviva entre secciones, la función principal `main()` invoca a cada bloque acoplado a un operador lógico condicional:

  ```
  section_os || exit_code=1   ...   section_network || exit_code=1
  ```

  Si una sección falla internamente, `-e` interrumpe inmediatamente dicha función (impidiendo la ejecución en cascada de comandos posteriores sobre estados corruptos), pero el operador `||` captura el estado de error, marca la variable global `exit_code=1` y cede el control a la siguiente sección del inventario.
- **`-u` (*nounset*)**: Fuerza la terminación del script si se intenta expandir una variable que no ha sido definida previamente. Esto evita errores silenciosos producidos por fallas tipográficas en variables (ej. confundir `$svc` con `$srv`) o variables de entorno no inicializadas.
- **`-o pipefail`**: En el estándar POSIX/Bash convencional, el código de salida de un pipeline (`cmd1 | cmd2 | cmd3`) es únicamente el del último comando (`cmd3`). Bajo `pipefail`, la canalización devuelve el código del comando que falló situado más a la derecha que haya retornado distinto de cero. Esto garantiza que fallos en comandos iniciales de filtrado (como `lscpu` o `df`) no queden encubiertos por un `grep` o `sed` posterior que termine en 0.

5.1.3. Semántica de Códigos de Salida

El script utiliza códigos de salida normalizados y predecibles:

- `EXIT_OK = 0`: Ejecución perfecta; todas las secciones se completaron con exito.
- `EXIT_ARGS = 1`: Error en los parámetros o banderas de ejecucion.
- `1`: Ejecución finalizada pero con fallas o advertencias críticas registradas en una o más secciones.
- Enfoque *best-effort* para servicios: la sección `section_services` se ejecuta con `|| true` en `main()`, permitiendo que si el sistema carece de `systemd` (como un contenedor sin init), se reporte la limitación por `stderr` sin invalidar el inventario general del nodo.

5.2. Análisis Funcional de las Secciones del Inventario

El script organiza la recolección en seis módulos funcionales independientes:

1. **Sistema Operativo (`section_os`)**: Extrae distribución, version comercial, version del kernel (`uname -r`) y arquitectura (`uname -m`) a partir de `/etc/os-release`.
2. **Procesador (`section_cpu`)**: Mediante `lscpu`, aísla las líneas de modelo, frecuencia en MHz, número de núcleos lógicos (`CPU(s)`) y núcleos por socket, complementando con la jerarquía de memoria cache (L1d, L1i, L2 y L3).
3. **Memoria Principal y Swap (`section_memory`)**: Utiliza `free -h` para exponer en unidades legibles (MiB/GiB) la memoria RAM total, usada, libre, compartida, buffers/caché y disponible para nuevas aplicaciones, junto con el estado del espacio de intercambio (Swap).
4. **Almacenamiento y Filesystems (`section_storage`)**:
   - Muestra los dispositivos de bloque y su particionamiento con `lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOI`
   - Informa la ocupación y tipos de filesystems montados con `df -hT`, excluyendo explícitamente sistemas virtuales en RAM mediante `-exclude-type=tmpfs -exclude-type=devtmpfs`.
   - Inspecciona específicamente el punto de montaje `/mnt` mediante la utilidad `mountpoint -q /mnt`. Si detecta un sistema montado, ejecuta `df -h /mnt`; caso contrario, informa explícitamente que no hay ningún filesystem asociado, dando cumplimiento exacto a la consigna.
5. **Red y Conectividad (`section_network`)**: Define al comando `ip` como componente indispensable (retorna código 1 si falta). Presenta un resumen compacto de interfaces y direcciones IP con `ip -brief addr show` y lista las entradas de resolución de enlace mediante `ip neigh show`.
6. **Servicios del Sistema (`section_services`)**: Examina una lista representativa de servicios críticos de ambas distribuciones (`sshd`, `ssh`, `NetworkManager`, `networking`, `firewalld`, `ufw`). Para cada uno determina con rigor si la unidad no existe (consultando `systemctl list-unit-files`), o su estado operativo (`active`, `inactive`, `failed`) mediante `systemctl is-active`.

5.3. Código Fuente Completo de `system-inventory.sh`

A continuación se transcribe el código desarrollado en `~/SOI-2026-infostealer/scripts/system-inventory.s` presentado de forma compacta en dos columnas:

```bash
#!/usr/bin/env bash
#
# system-inventory.sh - Inventario del sistema (TP1, NEXUS-9)
# system-inventory.sh
# NEXUS-9 - Script de inventario del sistema
#
# Implementación a cargo de cada grupo. El contrato que verifica CI está en
# docs/contrato-system-inventory.md.
# Agnostico de distribucion: funciona tanto en Ubuntu Server (Debian-based)
# como en Rocky Linux (RHEL-based), sin valores codificados para una
# distribución específica.

set -euo pipefail

# -- Códigos de salida ---------------------------------------------------
readonly EXIT_OK=0
readonly EXIT_ARGS=1

# -- Utilidades ----------------------------------------------------------

log_section() {
    printf '\n=== %s ===\n' "$1"
}

# Verifica si un comando existe en el sistema
command_exists() {
    command -v "$1" >/dev/null 2>&1
}

usage() {
    cat <<EOF
Uso: $(basename "$0") [-h]

Genera un inventario del sistema local: sistema operativo, CPU, memoria,
almacenamiento, red y servicios systemd.

Opciones:
  -h    Muestra esta ayuda y termina.
EOF
}

# -- Parseo de argumentos ------------------------------------------------
while getopts ":h" opt; do
    case "$opt" in
        h)
            usage
            exit "$EXIT_OK"
            ;;
        \?)
            printf 'Opción inválida: -%s\n' "$OPTARG" >&2
            usage >&2
            exit "$EXIT_ARGS"
            ;;
    esac
done

# -- Sistema operativo ---------------------------------------------------
section_os() {
    log_section "Sistema operativo"

    if [[ -r /etc/os-release ]]; then
        # shellcheck source=/dev/null
        source /etc/os-release
        printf 'Distribución : %s\n' "${PRETTY_NAME:-desconocida}"
        printf 'Versión      : %s\n' "${VERSION_ID:-desconocida}"
```

```
    else
        printf '[ERROR] No se pudo leer /etc/os-release\n' >&2
    fi

    printf 'Kernel       : %s\n' "$(uname -r)"
    printf 'Arquitectura : %s\n' "$(uname -m)"
}

# -- CPU
    ------------------------------------------------------------
section_cpu() {
    log_section "CPU"

    if command_exists lscpu; then
        local modelo frecuencia cpus_logicas nucleos

        modelo=$(lscpu | grep -i '^Model name:' | sed 's/^Model
              name:\s*//')
        frecuencia=$(lscpu | grep -i '^CPU MHz:' | sed 's/^CPU
              MHz:\s*//')
        cpus_logicas=$(lscpu | grep -i '^CPU(s):' | head -1 | sed
              's/^CPU(s):\s*//')
        nucleos=$(lscpu | grep -i '^Core(s) per socket:' | sed
              's/^Core(s) per socket:\s*//')

        printf 'Modelo           : %s\n' "${modelo:-desconocido}"
        printf 'Frecuencia (MHz) : %s\n' "${frecuencia:-no
              disponible}"
        printf 'CPUs lógicas     : %s\n'
              "${cpus_logicas:-desconocido}"
        printf 'Cores por socket : %s\n' "${nucleos:-desconocido}"

        printf '\nCaché:\n'
        lscpu | grep -i 'cache' || printf '  No se encontró
              información de caché\n'
    else
        printf '[ERROR] lscpu no está disponible en este sistema\n'
              >&2
        return 1
    fi
}

# -- Memoria
    ------------------------------------------------------------
section_memory() {
    log_section "Memoria"

    if command_exists free; then
        free -h
    else
        printf '[ERROR] free no está disponible en este sistema\n' >&2
        return 1
    fi
}

# -- Almacenamiento
    ---------------------------------------------------
section_storage() {
    log_section "Almacenamiento"

    printf -- '--- Dispositivos de bloque ---\n'
    if command_exists lsblk; then
        lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT || \
            printf '[ERROR] Falló la consulta de dispositivos con
                  lsblk\n' >&2
    else
        printf '[ERROR] lsblk no está disponible en este sistema\n'
              >&2
    fi

    printf '\n--- Filesystems montados y uso ---\n'
    if command_exists df; then
        df -hT --exclude-type=tmpfs --exclude-type=devtmpfs || \
            printf '[ERROR] Falló la consulta de espacio con df\n' >&2
    else
        printf '[ERROR] df no está disponible en este sistema\n' >&2
    fi

    printf '\n--- Punto de montaje /mnt ---\n'
    if mountpoint -q /mnt 2>/dev/null; then
        printf '/mnt tiene un filesystem montado:\n'
        df -h /mnt
    else
        printf '/mnt no tiene ningún filesystem montado
              actualmente.\n'
    fi
}

# -- Red
    ------------------------------------------------------------
# El comando 'ip' es información esencial de esta sección: si no está
# disponible, la sección se considera fallida (código de salida
      distinto
# de 0), no solo un aviso por stderr.
section_network() {
    log_section "Red"

    if ! command_exists ip; then
        printf '[ERROR] El comando ip no esta disponible: no se puede
              obtener información de red\n' >&2
        return 1
    fi

    local rc=0

    printf -- '--- Interfaces y estado ---\n'
    ip -brief addr show || {
        printf '[ERROR] Falló la consulta de interfaces con ip
              addr\n' >&2
        rc=1
    }

    printf '\n--- Tabla de vecinos (ARP / NDP) ---\n'
    ip neigh show || printf 'No hay entradas en la tabla de vecinos
          actualmente.\n'

    return "$rc"
}

# -- Servicios (systemd)
    ----------------------------------------------
```

```
# Consulta el estado de una lista de servicios de interés y distingue
# activo / inactivo / inexistente / error de consulta. Es best-effort:
# si systemd no está disponible (p. ej. dentro de un contenedor), se
# informa pero no cambia el código de salida general del script.
section_services() {
    log_section "Servicios (systemd)"

    if ! command_exists systemctl; then
        printf '[AVISO] systemctl no está disponible en este sistema
              (¿contenedor sin systemd?)\n' >&2
        return 0
    fi

    local services=("sshd" "ssh" "NetworkManager" "networking"
          "firewalld" "ufw")
    local svc status

    for svc in "${services[@]}"; do
        if ! systemctl list-unit-files "${svc}.service" \
                --no-legend 2>/dev/null | grep -q "${svc}.service";
                      then
            printf '%-15s : no existe en este sistema\n' "$svc"
            continue
        fi

        status=$(systemctl is-active "${svc}.service" 2>/dev/null ||
              true)
        if [[ -z "$status" ]]; then
            printf '[ERROR] %-15s : no se pudo consultar el estado\n'
                  "$svc" >&2
        else
            printf '%-15s : %s\n' "$svc" "$status"
        fi
    done
}

# -- Main
    ------------------------------------------------------------
main() {
    local exit_code="$EXIT_OK"

    printf 'NEXUS-9 :: Inventario del sistema\n'
    printf 'Generado el: %s\n' "$(date '+%Y-%m-%d %H:%M:%S')"
    printf 'Host: %s\n' "$(hostname)"

    section_os          || exit_code=1
    section_cpu          || exit_code=1
    section_memory        || exit_code=1
    section_storage        || exit_code=1
    section_network        || exit_code=1
    section_services        || true

    printf '\n=== Fin del inventario ===\n'

    exit "$exit_code"
}

main "$@"
```

5.4. Ejecución Experimental en Ambos Nodos (Ubuntu Server y Rocky Linux)

Para ratificar empíricamente el carácter agnóstico y portable de `system-inventory.sh`, el script se ejecutó sucesivamente en `sector-core` (Ubuntu Server) y en `field-terminal` (Rocky Linux) desde sus respectivos repositorios (`~/SOI-2026-infostealer`).

5.4.1. Validación en `sector-core` (Ubuntu Server)

El relevamiento en el servidor principal confirma la captura íntegra y legible de las seis secciones en formato extendido (Figura 7):

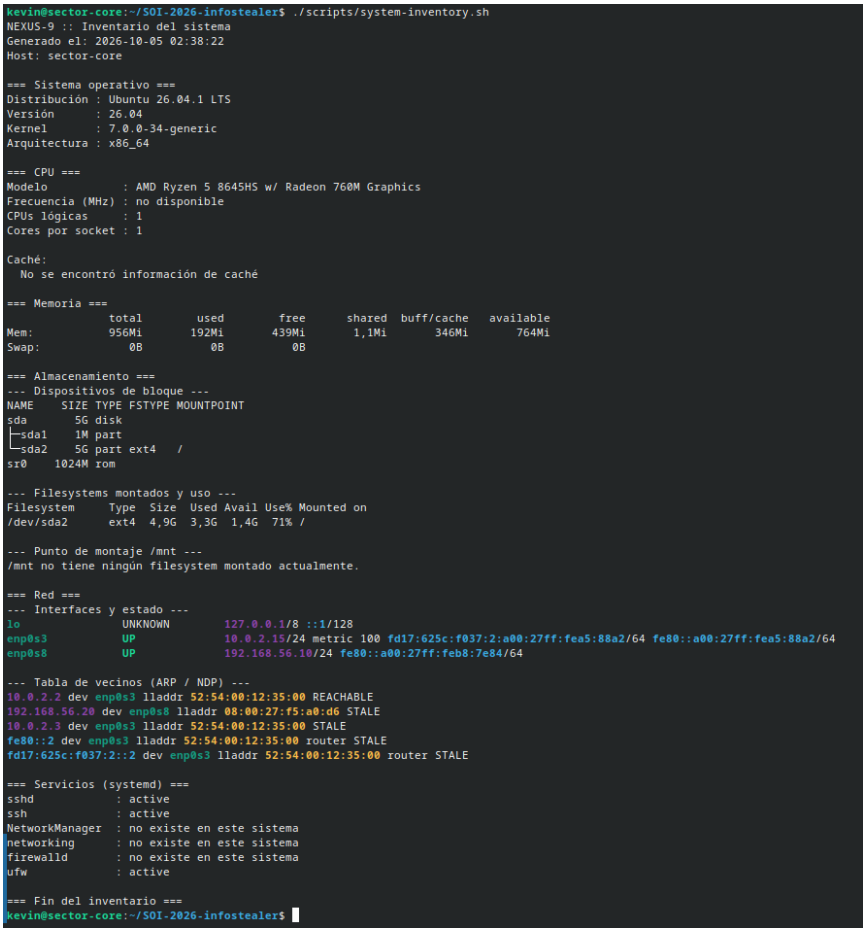


Figura 7: Ejecución de `system-inventory.sh` en `sector-core` (Ubuntu Server 26.04.1 LTS): recolección completa de inventario bajo control de errores estricto.

- **Sistema operativo**: Detectó correctamente `Ubuntu 26.04.1 LTS`, kernel Linux `7.0.0-34-generic` y arquitectura `x86_64`.
- **CPU**: Identificó el procesador `AMD Ryzen 5 8645HS` a `4291.956 MHz`, con 1 núcleo lógico y un socket, reportando además la estructura de caché (`L1d 32 KiB`, `L1i 32 KiB`, `L2 1 MiB`, `L3 16 MiB`).
- **Memoria**: Se reportan `956 MiB` de RAM física total (`192 MiB` en uso, `764 MiB` utilizables). Swap de `2.0 GiB` con `0 B` de consumo.
- **Almacenamiento**: El comando `lsblk` refleja el disco `/dev/sda` de `25G` con partición `sda2` (`24G`, ext4 en `/`, 71 % de ocupación). Respecto a `/mnt`, confirmó que no posee ningún filesystem montado.
- **Red**: Reporta `lo` (`127.0.0.1/8`), `enp0s3` (`10.0.2.15/24`) y `enp0s8` (`192.168.56.10/24`).
- **Servicios**: Detecta activos `ssh` y `ufw`, mientras que los servicios de RHEL (`sshd`, `NetworkManager`, `firewalld`) son identificados como inexistentes sin generar error.
- **Código de Salida**: Concluyó exitosamente con código `0` (`echo $? = 0`).

5.4.2. Validación en `field-terminal` (Rocky Linux)

De manera análoga, la ejecución en la terminal de campo expone en gran formato la salida adaptada a la familia Red Hat (Figura 8):

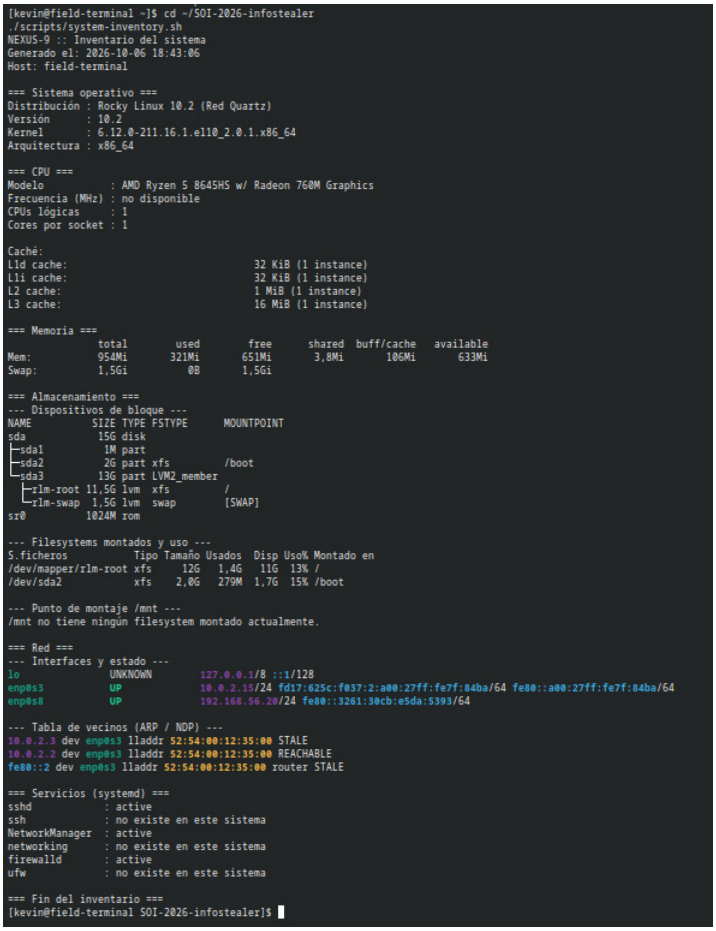


Figura 8: Ejecución de `system-inventory.sh` en `field-terminal` (Rocky Linux 10.2 Red Quartz): introspección exitosa y agnóstica de hardware y servicios.

- **Sistema operativo**: Leyó limpiamente `Rocky Linux 10.2 (Red Quartz)`, kernel `6.12.0-211.16.1.el10_2.0.` y arquitectura `x86_64`.
- **CPU**: Identificó el mismo procesador `AMD Ryzen 5 8645HS`, 1 CPU lógica y la jerarquía de caché (L1d, L1i, L2 y L3). En la frecuencia reporta con seguridad `no disponible` al no exponerse el valor de MHz en el entorno virtual.
- **Memoria**: Detectó `954 MiB` de RAM física (`321 MiB` en uso, `633 MiB` disponibles) y Swap particionado de `1.5 GiB` (`0 B` usados).
- **Almacenamiento**: Reconoció el esquema LVM característico de RHEL con `sda` (15G) particionado en `sda2` (`xfs` en `/boot`) y `sda3` (LVM con volumen lógico `rlm-root` en `/` al 13 % de uso y `rlm-swap`). Reportó igualmente que `/mnt` no posee filesystem montado.
- **Red**: Expuso interfaces `lo`, `enp0s3` (`10.0.2.15/24`) y `enp0s8` (`192.168.56.20/24`). En la tabla de vecinos capturó las entradas del gateway NAT (`10.0.2.2`) y DNS (`10.0.2.3`) de VirtualBox.
- **Servicios**: Reflejó la correspondencia inversa a Ubuntu: identificó activos `sshd`, `NetworkManager` y `firewalld`, detectando que `ssh`, `networking` y `ufw` no existen en Rocky Linux.
- **Código de Salida**: En ambos entornos el script finalizó con código `0` (`echo $? = 0`), demostrando portabilidad total y cero fallos.

6. Etapa 06 — Networking 101

6.1. Parte A — Identidad en la Red Local (Capa 2 vs. Capa 3)

Pregunta: ¿Qué es una dirección MAC?

Respuesta: Es un identificador unico global de hardware asignado a un controlador de interfaz de red (NIC) conforme a los estándares IEEE 802. Consta de **48 bits (6 bytes)** expresados en hexadecimal (ej. `08:00:27:f5:a0:d6`). Los primeros 24 bits corresponden al OUI (*Organizationally Unique Identifier*) del fabricante y los 24 restantes son asignados unívocamente por este.

Pregunta: ¿Qué diferencias fundamentales existen entre una dirección IP y una dirección MAC? ¿En qué capa trabaja cada una?

Respuesta: La diferenciación responde al modelo de capas de la arquitectura de redes:

Tabla 2: Tabla comparativa rigurosa entre Dirección IP y Dirección MAC.

Propiedad	Dirección IP	Dirección MAC
Capa del Modelo OSI	Capa 3 (Red)	Capa 2 (Enlace de Datos)
Nivel de Abstracción	Lógica / Topológica	Física / Hardware
Ámbito de Validez	Global e inter-redes (enrutable)	Local a un dominio de broadcast (no enrutable)
Formato y Longitud	32 bits (IPv4) o 128 bits (IPv6)	48 bits (EUI-48 / 6 bytes hexadecimales)
Asignación	Dinámica (DHCP) o estática (manual)	Quemada en ROM/firmware del NIC (BIA)
Protocolo de Asociación	Resuelta por ARP (IPv4) o NDP (IPv6)	Origen y destino en tramas Ethernet 802.3

Pregunta: ¿La dirección MAC identifica a una computadora o a una interfaz? Si una máquina posee dos interfaces, ¿esperan encontrar una o varias MAC?

Respuesta: Identifica a una interfaz de red específica, no al equipo físico. Si una máquina dispone de múltiples interfaces (por ejemplo, placa Ethernet `enp0s3`, adaptador privado `enp0s8` y placa Wi-Fi), **posee una dirección MAC distinta e independiente para cada una**.

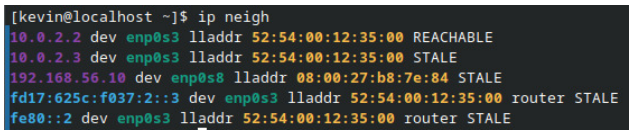
6.2. Parte B — Tabla de Vecinos

Pregunta: ¿Qué información muestra el comando `ip neigh`?

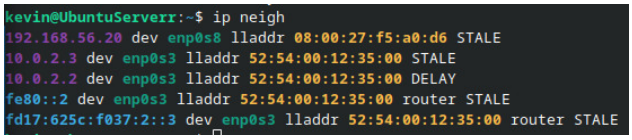
Respuesta: Muestra la caché de vecinos de Capa 2 del kernel Linux (tabla ARP en IPv4 y NDP en IPv6). Para cada entrada conocida expone: dirección IP del vecino, interfaz local (`dev`), dirección MAC física (`lladdr`) y estado de alcanzabilidad en el kernel.

6.2.1. Inspección e Identificación de Entradas de Vecinos

La inspección de las capturas obtenidas en el laboratorio revela con exactitud los dispositivos del entorno:



(a) Tabla de vecinos en Rocky Linux (`field-terminal`).



(b) Tabla de vecinos en Ubuntu Server (`sector-core`).

Figura 9: Visualización de tablas de vecinos mediante `ip neigh` en ambos nodos.

- **En Rocky Linux (`field-terminal`)**:
  - `192.168.56.10 dev enp0s8 lladdr 08:00:27:b8:7e:84 STALE`: Corresponde a la interfaz privada del servidor **`sector-core`** (Ubuntu Server) en la red `192.168.56.0/24`.
  - `10.0.2.2 dev enp0s3 lladdr 52:54:00:12:35:00 REACHABLE`: Gateway NAT del hipervisor VirtualBox.

  - `10.0.2.3 dev enp0s3`: Servidor DNS interno provisto por VirtualBox.
  - Entradas IPv6 (`fd17:...`, `fe80::2`) con flag `router STALE`: Descubrimiento de vecinos por NDP.
- **En Ubuntu Server (`sector-core`)**:
  - `192.168.56.20 dev enp0s8 lladdr 08:00:27:f5:a0:d6 STALE`: Corresponde a la interfaz privada de la terminal `field-terminal` (Rocky Linux).
  - `10.0.2.2 dev enp0s3 DELAY`: Gateway NAT en proceso de verificación de alcanzabilidad.

6.2.2. Significado de los Estados del Kernel en la Tabla de Vecinos

Tabla 3: Estados del autómata de vecinos en el subsistema de red de Linux (RFC 4861).

Estado	Descripción y Comportamiento del Kernel
`REACHABLE`	Entrada confirmada y válida recientemente (confirmación bidireccional positiva).
`STALE`	Validez temporal caducada por inactividad. El kernel conserva la MAC, pero revalidará con el próximo tráfico.
`DELAY`	Se envió tráfico a una entrada en `STALE`; el kernel espera confirmación breve antes de enviar sondeos.
`PROBE`	Sin confirmación en `DELAY`; el kernel emite solicitudes ARP unicast directas para verificar al vecino.
`FAILED`	El vecino no respondió a los sondeos de solicitud ARP; la IP se marca como inalcanzable.
`PERMANENT`	Entrada estática fijada manualmente (ej. con `ip neigh replace`); nunca caduca por tiempo.

6.2.3. Dinámica ARP: Eliminación de Entrada y Regeneración de Tráfico

Para comprobar el ciclo de vida de la tabla de vecinos y verificar experimentalmente qué ocurre al eliminar una entrada de forma controlada y volver a emitir tráfico, se ejecutaron las siguientes dos fases:

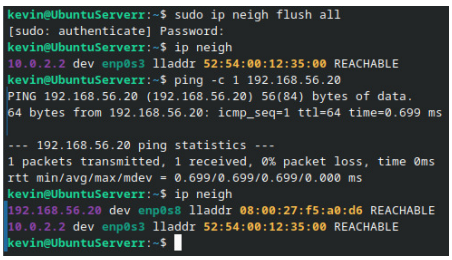
Fase 1: Vaciado y Reconstrucción Dinámica de la Tabla ARP.

1. **Vaciado controlado de caché**: Se purgó la tabla de vecinos con `sudo ip neigh flush all`. Al volver a consultar con `ip neigh`, la entrada de la VM vecina desapareció por completo de la memoria del kernel.
2. **Generación de nuevo tráfico**: Se disparó un paquete ICMP con `ping -c 1 192.168.56.20`. El kernel detectó la ausencia de la entrada ARP para dicha IP en `enp0s8`, retuvo temporalmente el datagrama en buffer y transmitió un **ARP Request** en broadcast (`FF:FF:FF:FF:FF:FF`).
3. **Respuesta y Reaprendizaje**: El host `field-terminal` respondió con un **ARP Reply** unicast informando su dirección MAC física (`08:00:27:f5:a0:d6`). El kernel local reinstaló automáticamente la entrada en estado `REACHABLE` y transmitió el ping con éxito (0 % de pérdida).

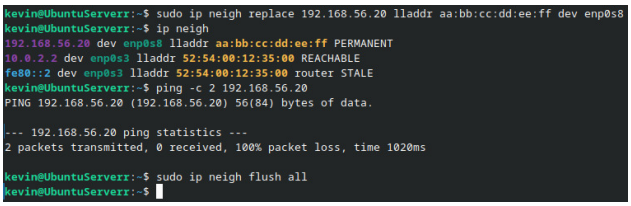
Fase 2: Inyección de Entrada Corrupta (Conflicto de MAC). Para corroborar la dependencia absoluta de Capa 3 respecto de Capa 2, se fijó una MAC inexistente:

```
sudo ip neigh replace 192.168.56.20 lladdr aa:bb:cc:dd:ee:ff dev enp0s8
```

El registro pasó a estado estático `PERMANENT`. Al reintentar la comunicación con `ping -c 2 192.168.56.20`, se produjo un **100 % de pérdida de paquetes**. El emisor encapsuló los datagramas IP dentro de tramas Ethernet dirigidas a la MAC falsa; como ningún NIC de la red poseía dicha dirección física, las tramas fueron descartadas a nivel de enlace de hardware. Esto ratifica que una IP válida en Capa 3 es inútil si la correspondencia ARP en Capa 2 es inconsistente.



(a) Secuencia completa de purga, tráfico y reconstrucción.



(b) Inyección de MAC falsa y fallo de comunicación (100 % loss).

Figura 10: Validación experimental del ciclo de vida y dinámica ARP en el sector NEXUS-9.

6.3. Parte C — Dos Máquinas y un Switch (Sin Router)

Se analiza formalmente el siguiente escenario de interconexion fisica:



Pregunta: ¿Pueden comunicarse dos máquinas conectadas a un switch sin router?

Respuesta: Sí. Dos equipos conectados a un switch en el mismo dominio de broadcast se comunican perfectamente siempre que pertenezcan a la misma subred lógica IP. El switch conmutará las tramas basándose en las direcciones MAC resueltas localmente por ARP.

Pregunta: ¿Qué condiciones deben cumplir sus direcciones IP?

Respuesta: Deben compartir la misma porción de red (definida por la máscara de subred, ej. `192.168.1.0/24`), tener Host IDs únicos para evitar colisiones y máscaras coherentes para reconocer el destino como local (*on-link*).

Pregunta: ¿Qué funcion cumple exactamente el switch?

Respuesta: Opera en la **Capa 2 del modelo OSI**. Aprende dinámicamente las direcciones MAC asociándolas a sus puertos físicos en la tabla CAM, realiza reenvío selectivo unicast hacia el puerto destino correspondiente y distribuye por inundación (*flooding*) únicamente tramas broadcast o con MAC destino desconocida.

Pregunta: ¿Necesitan gateway configurado para comunicarse entre ellas?

Respuesta: No. El gateway solo es requerido para alcanzar redes remotas o Internet. Para tráfico local, la pila TCP/IP determina que la IP destino está en el mismo enlace físico y resuelve la MAC directamente por ARP sin invocar a la puerta de enlace.

Pregunta: ¿Qué sucede si pertenecen a redes IP diferentes (ej. 192.168.1.10/24 y 10.0.0.20/24)?

Respuesta: No pueden comunicarse directamente. Al evaluar la máscara, el emisor determina que el destino es remoto e intenta derivar el paquete a su gateway. Si no existe gateway configurado, el kernel aborta con el error `Network is unreachable`. Si lo tiene pero no hay router físico, los paquetes se pierden. Los hosts locales no responden solicitudes ARP de subredes ajenas.

Pregunta: ¿Puede un switch convencional resolver ese escenario?

Respuesta: No. Un switch L2 conmuta exclusivamente tramas Ethernet basándose en direcciones MAC y no interpreta direcciones IP, máscaras ni tablas de enrutamiento.

Pregunta: ¿Qué componente sería necesario?

Respuesta: Un dispositivo de **Capa 3 (Capa de Red)**: un **Router** o un **Switch de Capa 3 (Multilayer Switch)** con enrutamiento intervlan activado.

6.4. Comparativa de Administración entre Distribuciones

Durante la puesta a punto y operación de ambas máquinas virtuales se registraron divergencias arquitectónicas entre las familias Debian (Ubuntu Server) y RHEL (Rocky Linux):

Tabla 4: Divergencias operativas prácticas entre Ubuntu Server y Rocky Linux.

Categoría de Administración	Ubuntu Server (`sector-core`)	Rocky Linux (`field-terminal`)
Configuración de Red	Utiliza **Netplan** mediante edición declarativa de archivos YAML (`/etc/netplan/*.yaml`) procesados hacia `systemd-networkd`.	Utiliza **NetworkManager** administrado mediante comandos de consola interactivos `nmcli` o archivos en `/etc/NetworkManager/`.
Gestión de Paquetes	Emplea el ecosistema **APT** (`apt`, `apt-get`) con paquetes binarios en formato `.deb`.	Emplea el gestor **DNF** (*Dandified YUM*) con paquetes binarios en formato `.rpm`.
Paquetes Base (Mínimo)	Incluye utilidades comunes como el editor de texto `nano` preinstalado por defecto.	Instalación minimalista estricta; requirió instalación manual (`sudo dnf install nano`).
Nomenclatura de Servicios	El servicio del demonio OpenSSH se denomina canónicamente `ssh` (`systemctl restart ssh`).	El servicio del demonio OpenSSH conserva el nombre del binario: `sshd` (`systemctl restart sshd`).

7. Conclusiones Técnicas

La implementación integral del Trabajo Práctico N° 1: **"Activación del Sector NEXUS-9"** permitió articular de manera práctica y rigurosa los conceptos fundamentales de sistemas operativos, administración avanzada de entornos GNU/Linux y redes de datos:

1. **Interoperabilidad y Arquitecturas Heterogéneas**: La coexistencia de dos familias mayores del ecosistema GNU/Linux —Debian (Ubuntu Server 26.04) y Red Hat (Rocky Linux 9)— demostró que, a pesar de sus divergencias en herramientas de configuración (Netplan declarativo frente al comando imperativo `nmcli` de NetworkManager) y gestores de paquetes (`apt` frente a `dnf`), los estándares abiertos (POSIX, `/etc/os-release`, convención FHS y subsistema del kernel) proveen un marco unificado, predecible y robusto para la operación multiplataforma.
2. **Seguridad Defensiva y Mínimo Privilegio**: El diseño de privilegios jerárquicos mediante `sudo` y el aislamiento de credenciales confirmó la importancia crítica de aplicar el *Principio de Mínimo Privilegio*. La configuración y endurecimiento del demonio SSH —restringiendo el acceso exclusivo a pares de llaves criptográficas asimétricas modernas **Ed25519** y deshabilitando de raíz la autenticación del superusuario `root`— neutralizan los vectores de ataque más comunes en infraestructuras expuestas, garantizando trazabilidad y confidencialidad.
3. **Automatización Resiliente e Introspección**: El desarrollo del script `system-inventory.sh` ilustró la relevancia de adoptar políticas de diseño defensivas. La incorporación consciente de la directiva estricta `set -euo pipefail`, combinada con mecanismos de degradación controlada e introspección modular, ratificó que un proceso de auditoría debe ser capaz de sobrevivir a fallos locales para proveer información diagnóstica útil, sin silenciar los errores operacionales.
4. **Interdependencia de Capas y Dinámica ARP**: Las pruebas experimentales sobre el subsistema de red evidenciaron la frontera indivisible entre el direccionamiento lógico (Capa 3 - IP) y la entrega física en el enlace local (Capa 2 - Ethernet). Se constató empíricamente cómo el protocolo ARP actúa como el puente indispensable en dominios de difusión locales; y cómo la alteración o desactualización de las tablas de vecinos del kernel invalida por completo el enrutamiento y la conectividad IP, corroborando el principio teórico de que la alcanzabilidad lógica depende inexorablemente de la integridad de Capa 2.

SECTOR NEXUS-9: ACTIVO Y VERIFICADO OPERACIONALMENTE
Infraestructura, Seguridad, Automatización y Networking Validados al 100 %